SESSION 4: MODULES, LOOPS, AND CODE REUSABILITY

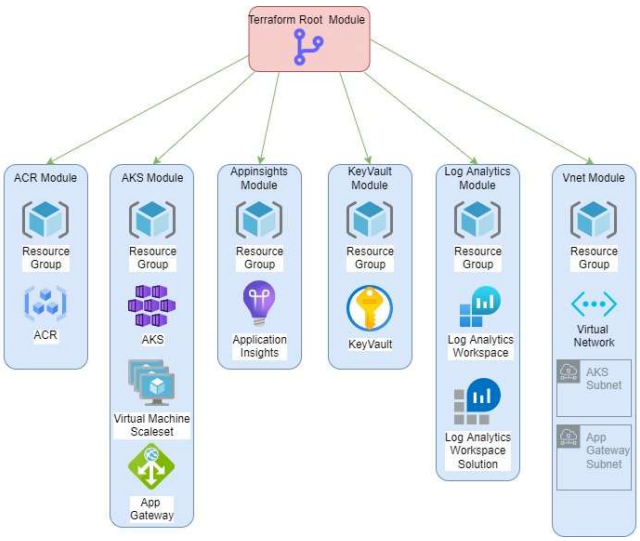
Interno

Terraform Modules

- A **Terraform module** is a reusable, self-contained set of Terraform configuration files
- Encapsulates multiple resources into a logical unit
- Promotes **reuse, consistency, and standardization**
- Enables composition of complex infrastructures
- Used for:
  - Environments (dev, qa, prod)
  - Common building blocks (networking, compute, security)

Key benefits

- DRY principle (Don't Repeat Yourself)
- Easier maintenance and refactoring
- Improved collaboration across teams
- Faster and safer infrastructure delivery



Fonte: https://peerdh.com/blogs/programming-insights/best-practices-for-building-and-managing-terraform-modules

Interno

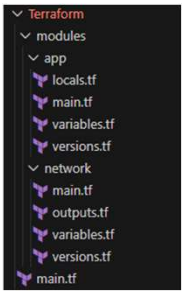
Module Structure, Inputs & Outputs

Key components

- main.tf – resources definition
- variables.tf – input variables
- outputs.tf – exposed values
- versions.tf – provider and Terraform constraints

Inputs & Outputs

- Inputs = parameters passed into the module
- Outputs = values returned by the module
- Enable loose coupling between modules




```
provider "azurerm" {
  features {}
}

module "network" {
  source = "./modules/network"

  resource_group_name = "rg-demo"
  location            = var.region
  vnet_name           = "vnet-demo"
  address_space = ["10.0.1.0/26"]
  subnet_name         = "subnet-app"
  subnet_prefix = ["10.0.1.0/28"]

}

module "webapp" {
  source = "./modules/webapp"


  app_name            = "webapp-demo"
  location            = "East US"
  resource_group_name = "rg-demo"


  subnet_id = module.network.subnet_id
}
```

FINDMORE ACADEMY

Interno

53

Scope of Variables on Modules

- **Input Variables:** Defined in a module (variables.tf) to parameterize the module. They make modules reusable and configurable. This is how modules accept external data.
- **Parent Module Variables:** Variables declared in the root module (where you call child modules) can be passed to child modules as inputs.
- **Child Module Variables:** Variables inside a module that are set via the parent module's module block. Child modules do **not** automatically inherit parent module variables—they must be explicitly passed.
- **Local variables** inside a module are for internal computation and do not require inputs.
- **Outputs** expose module values back to the parent module

Terraform > modules > network > variables.tf > ...

```
# The child module has its own variables.tf to define what inputs it expects.
variable "resource_group_name" {
  type        = string
  description = "Resource Group name"
}

variable "location" {
  type        = string
  description = "Azure region"
}

variable "vnet_name" {
  type        = string
}

variable "address_space" {
  type        = list(string)
}

variable "subnet_name" {
  type        = string
}

variable "subnet_prefix" {
  type        = list(string)
}
```

Terraform > main.tf > ...

```
# The parent module declares region (internal
# to parent) and passes the other parameters
# explicitly to the child module.
variable "region" {
  default = "West Europe"

}

module "network" {
  source = "./modules/network"

  resource_group_name = "rg-demo"
  location            = var.region
  vnet_name           = "vnet-demo"
  address_space = ["10.0.1.0/26"]
  subnet_name         = "subnet-app"
  subnet_prefix = ["10.0.1.0/28"]

}
```

FINDMORE ACADEMY

Interno

54

Module Sourcing

- Defines where Terraform retrieves a module from when it is referenced in a module block.
- Terraform supports multiple source types, enabling reuse and sharing of infrastructure code.

Local path
- Used for development or tightly coupled modules
- Example: ./modules/network

Terraform Registry
- Public or private, versioned, and documented
- Example: hashicorp/aws/vpc

Git repositories
- Supports GitHub, GitLab, Bitbucket, etc.
- Can reference branches, tags, or commits

Remote archives
- ZIP or TAR files over HTTPS
- Useful for packaged modules

```
# Terraform Registry module
module "vpc" {
  source  = "terraform-aws-modules/vpc/aws"
  version = "~> 5.0"
}


# Git-based module
module "network_git" {
  source = "git::https://github.com/org/network-module.git?ref=v1.2.0"
}


# Local Module
module "webapp" {
  source = "./modules/webapp"


  app_name            = "webapp-demo"
  location            = "East US"
  resource_group_name = "rg-demo"


  subnet_id = module.network.subnet_id
}
```

https://registry.terraform.io/browse/modules

Interno

Module Versioning

Why Version Modules?

- Ensures **repeatable and predictable** infrastructure deployments
- Prevents breaking changes from upstream updates
- Enables controlled upgrades and rollback

Versioning modes

- Terraform Registry:

version = "~> 5.0"

- Git-based modules:

source = "git::https://github.com/org/module.git?ref=v1.2.0"

```
# Terraform Registry module
module "vpc" {
  source  = "terraform-aws-modules/vpc/aws"
  version = "~> 5.0"
}


# Git-based module
module "network_git" {
  source = "git::https://github.com/org/network-module.git?ref=v1.2.0"
}
```

Interno

When to Use a Module vs a Resource

Use a Resource When:

- You are provisioning a single infrastructure object
- The configuration is simple and unlikely to be reused
- You are experimenting or prototyping

Use a Module When:

- Multiple resources form a logical unit (e.g., VPC + subnets + routes)
- The pattern is reused across environments or teams
- You want to standardize infrastructure implementation

If you copy-paste more than one resource block, it should probably be a module.

Interno

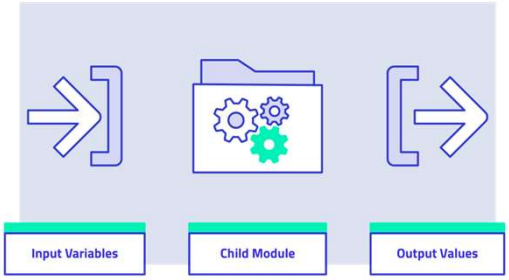
Module Design Best Practices

Do

- Keep modules **small and focused**
- Define **clear inputs and outputs**
- Use **sensible defaults** for variables
- Document module usage and assumptions

Don't

- Hardcode environment-specific values
- Create tightly coupled modules
- Expose unnecessary outputs
- Mix unrelated resources in one module



https://github.com/terraform-docs/terraform-docs

Interno

Loops in Terraform: Count

Count creates multiple instances of a resource or module based on a numeric value.

Best Used When:

- You need a **fixed number** of identical resources
- Resources are **position-based**, not identity-based
- Order does not matter beyond indexing

```
# example of count
resource "azurerm_windows_virtual_machine" "vm" {
  count               = 3

  # Name constructed using count and prefix
  name                = "windows-vm-${count.index}"
  resource_group_name = azurerm_resource_group.rg.name
  location            = var.location
  size                = "Standard_F2"
  admin_username      = var.vm_admin_login
  admin_password      = var.vm_admin_password
  tags = var.tags
  network_interface_ids = [
    azurerm_network_interface.nic[count.index].id ]
 # allow linking the VMs with the corresponding network interface


  os_disk {
    caching              = "ReadWrite"
    storage_account_type = "Standard_LRS"
  }

}
```

FINDMORE ACADEMY

Interno

59

Loops in Terraform: for_each

for_each creates resources from a map or set, using keys as stable identifiers.

Best Used When:

- Each resource has a **distinct identity**
- You want **stable addressing** over time
- You need to add/remove items safely

```
variable "storage_accounts" {
  description = "List of Storage Account names"
  type        = list(string)
  default     = ["stterraform01", "stterraform02"]
}

# Storage Accounts using loop (for_each)
resource "azurerm_storage_account" "storage" {
  for_each = toset(var.storage_accounts)

  name                     = each.value
  resource_group_name      = azurerm_resource_group.rg.name
  location                 = azurerm_resource_group.rg.location
  account_tier             = "Standard"
  account_replication_type = "LRS"


  tags = {
    environment = "demo"
  }
}
```

```
# example of for_each loop
variable "instances" {
  type = map(string)
  default = {
    web  = "t3.micro"
    api  = "t3.small"
  }
}

resource "aws_instance" "example"
{
  for_each      = var.instances
  ami           = "ami-123456"
  instance_type = each.value

  tags = {
    Name = each.key
  }
}
```

If resources have names, roles, or meanings → use for_each.

Interno

60

Conditional Logic: ("if")

Terraform **does not have** traditional if-statements, but uses **conditional expressions** (condition ? true : false) to control values and behavior.

Best Used When:

- You need **optional behavior**
- You want to enable/disable configuration dynamically
- Combined with "count" or "for_each" for conditional creation

```
variable "create_bucket" {
  description = "Whether to create the S3 bucket"
  type        = bool
  default     = false
}

# Terraform evaluates the conditional expression:
# If var.create_bucket is true → count = 1 → one S3 bucket is created
# If var.create_bucket is false → count = 0 → no S3 bucket is created at all

resource "aws_s3_bucket" "example" {
  count  = var.create_bucket ? 1 : 0
  bucket = "my-example-bucket"
}


# another example
instance_type = (var.env == "prod") ? "t3.large" : "t3.micro"
```

Interno

Loops in Terraform: for

for expressions transform one collection into another.

Best Used When:

- You need to **derive values**
- You want to reshape inputs for for_each
- You need filtering or mapping logic

```
variable "subnets" {
  type = list(object({
    name = string
    cidr = string
    public = bool
  }))

  default = [
  { name = "web", cidr = "10.0.1.0/24", public = true },
  { name = "db",  cidr = "10.0.2.0/24", public = false }
  ]

}

locals {
  public_subnets = {
    # filtering the list and converting into a map
    for subnet in var.subnets :
    subnet.name => subnet.cidr
    if subnet.public
  }
}

# the new map variable can be used on a for_each
resource "aws_subnet" "public" {
  for_each = local.public_subnets


  vpc_id     = aws_vpc.main.id
  cidr_block = each.value


  tags = {
    Name = each.key
  }
}
```

Interno

Loops in Terraform: for

- The type of brackets around the **for** expression decide what type of result it produces.
- [ ] produces a tuple.
- { } produces an object, so you must provide two result expressions that are separated by the **=>** symbol:

```
# Deriving an object from a list
{for s in var.list : s => upper(s)}


# Result:
{
  foo = "FOO"
  bar = "BAR"
  baz = "BAZ"
}
```

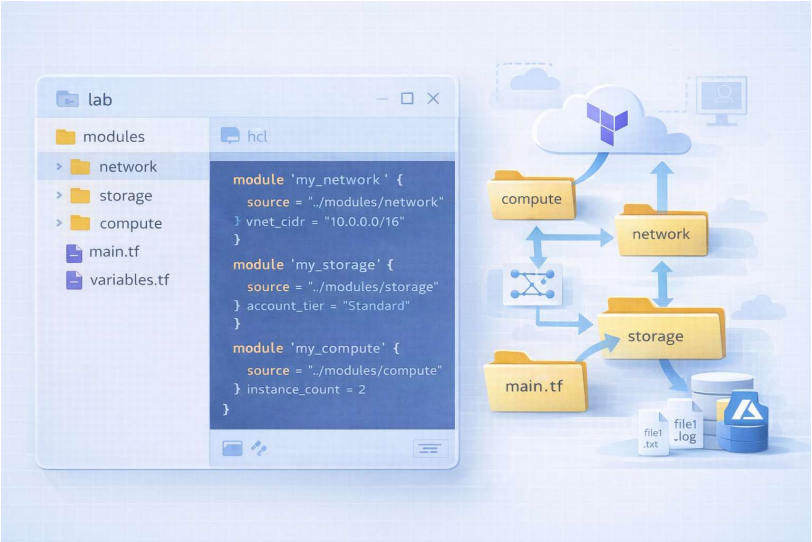
```
variable "regions" {
  description = "List of Azure regions"
  type        = list(string)

  default = [
    "westeurope",
    "northeurope",
    "canadacentral",
    "eastus",
    "norwayeast"
  ]
}

locals {
  regions_with_europe = [
    # creates a new tuple (list) containing only regions
    # that contains the word "europe".
    for region in var.regions : region
    if contains(region, "europe")
  ]
}
```

Interno

Lab 4: Building a Reusable Module



Interno